UTM

UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER 2 2023/2024

Project Deliverable 2 - Problem Analysis and Design

SECJ 1023 - Programming Technique II

SECTION 03

LECTURER: DR. JUMAIL BIN TALIBA

NAME	MATRIC NUMBER
NURUL IKA SYAFINY BINTI AZHAR	A23CS0164
NURAISYAH BINTI MOHD ZIKRE	A23CS0160
MOHAMED ALIF FATHI BIN ABDUL LATIF	A23CS0112
MUHAMMAD AFIQ DANIAL BIN ROZAIDIE	A23CS0117

1.0 Introduction

"Math Battle" is a game where the player will need to do calculations in order to win the game. Our game was inspired from the game "Battle Arena: RPG Adventure". The game only requires one player to play at a time and as mentioned they have to fight mathematical battles strategically in order to win the game. The player will have a limited life as they have to defeat the enemy by carefully choosing their opponents. The player may lose the strength after defeating the enemy but they also can gain strength from the loot in the game.

The game will start by a player having a small number of lives and they need to increase it by choosing loot. Not to mention that the player needs to be careful in choosing the enemy. This is said because if the player chooses the enemy with bigger lives than the player, the lives of the player will decrease. Therefore, the tips to the game are to defeat weaker opponents and choose loot to increase power. In order to get the game getting more intense and fun, the player needs to have a good strategy by thinking quickly to choose a correct opponent or loot.

In order to implement this game, we have done brainstorming sessions within the group members and our lecturer. Hence, we came to conclude by applying multiple Object-Oriented Programming (OOP) concepts which consist of encapsulation, composition, inheritance and polymorphism. These concepts can be found through our Unified Modelling Language (UML) diagram.

2.0 Problem Analysis

In order to identify classes and objects needed in our game, a brainstorming session was held to discuss more about our game needs. Below is the recording of our brainstorming session that was held:

1. First Discussion (26/5/2024): https://drive.google.com/file/d/1mr9XeuJwuomNbP5t2WAS63bM83xrftto/view?usp=sharing

2. Second Discussion (4/6/2024): https://drive.google.com/file/d/1oGZ7mJWrLK6g2YcE5_G23gKy0mRxJCpx/view?usp=sharing

a. Object and classes involved in the project

1. Class Background

- gameBG : string
- gameOverBG : string
- gameWinBG : string
+ Background (): void
+ setBG(): void
+ setGameOverBG(bool): void
+ setGameWinBG(): void
+ gameOverBackground(l) : void
+ gameWinBackground() : void
+ drawGameBG() : void
+ undrawBG() : void

This class which is Class Background will set a background as decoration within the game and also will change if the player wins or loses the game.

2. **Class Game**

- gameStat: bool
- clickX: float
- click Y: float
+ player: Player
+ loot: Loot
+ enemy[ ]: Enemy
+ gameJourney(): void
+ setPlayerHealth(int) : void
+ detectClick(): void
+ drawBackground(): void
+ decideWinner(int,int) : bool
+ returnNewHealth(): int

This class Game will conduct as a brain for the whole game including, calculate the health and also decide if the player wins the game or not.

3. **Class ClickBox**

- width : float
- height : float
- x : int
- y : int
+ boxsixe: Element
+ ClickBox() : ClickBox
+ setSize (int, int, float, float)
+ drawBox() : void
+ undrawBox() : void
+ ~ ClickBox()

Class ClickBox will draw an invisible box around the opponent for the player to click and proceed to the next round.

4. **Class Element**

- elementPic : char
- positionX : int
- positionY : int
- width : float
- length : float
+ Element() : Element
+ setElementPic (char) : void
+ setPosition (int, int, float, float) : void
+ drawElement() : void
+ undrawElement() : void
+ getX() : int
+ getY() : int
+ getWidth() : float
+ getLength() : float

Class Element will set and display the correct size an image of a character whether player and opponent so it will be visible to the player.

5. **Class Score**

    - playerHealth : int
    - opponentHealth : int
    - lootHealth : int
    + element: Element
    + Score (): Score
    + getPlayerHealth(int): void
    + getOpponentHealth(int): void
    + getLootHealth(int): void
    + drawPlayerHealth (int, int) : void
    + drawOpponentHealth (int, int) : void
    + drawLootHealth (int, int) : void
    + undrawHealth() : void

Class Score will draw or undraw the health of the player and opponent.

6. **Class Player**

    - health: int
    + Player(): Player
    + setHealth(int): void
    + getHealth(): int

Class Player will set the health of a player and send it to Class Game.

7. **Class Opponent**

    - level: int
    + Opponent(): Opponent
    + setLevel(int): void
    + rand(): int
    + getLevel(): int
    + ~Opponent(): Opponent

Class Opponent is the as a Class Player except the number of health will be randomized.

8. **Class Enemy**

    - numOfEnemy : int
    - enemyScore : int
    + Enemy() : Enemy
    + winnerHp(int): int
    + ~Enemy()

Class Enemy will compare the health of the player and Enemy then return the result to the Class Game.

9. **Class Loot**

    - lootScore : int
    + Loot() : Loot
    + winnerHp(int): int
    + ~Loot()

Class Loot is more or less same as the Class Enemy and the only difference is the health of the loot added to the player as a bonus.

b. Class relationships

Once we determine all the needed objects and classes for our project, we decide the relationship between the classes.

1. *Association relationship*

I. Class Background with class Game

Class Game is associated with class Background because the function decideWinner in the class Game will change the background of the game window if the player wins, loses or is still playing the game. Class Game will have access to properties in the class Background whenever the change of background is needed.

II. Class Game with Class ClickBox

Class Game needs access to class ClickBox in order to detect with which element the player clicks in the detectClick function. This function will take the coordinate of the object box created by ClickBox class and compare it with the player's click input to determine which box is clicked by the player.

III. Class Player and Class Score and Class Opponent

In order for class Score to create an object that displays player health and opponent level, it needs to gain player health and opponent level from the class Player and class Opponent respectively. In this case, we use association to let class Score gain access to the member in class Player.

2. *Composition relationship*

I. Class Game with class Enemy , class Loot and class Player

All the three classes rely on class Game because if the game is not started, no player, enemy or loot will be created. Class Game is the one that starts the progress of the game till the end. Once the game is over, all the elements will be deleted as well.

II. Class Element with Class ClickBox

In this case, ClickBox objects will not be created if the element object is not present in the game. This is because the box created by ClickBox objects rely on the size and coordinate of the element created first. The box that will be shaped by the class will be the same size and position as the element. Once the class Element object is destructed, the object created by class ClickBox will also be destructed.

3. *Inheritance relationship*

I. Class Player and Class Opponent inherited to Class Element

Class Player and Class Opponent as the child inherit with Class Element as the parent allow them to access all attributes and methods in the parent class. In this case, Class Player and Opponent needs attributes like positionX and positionY to allocate the coordinate for them and attributes like width and length to set the size of the picture for player and opponent.

II. Class Enemy and Class Loot inherited to Class Opponent

For this case, Class Enemy and Loot as child are inherited to Class Opponent as parent because Class Enemy and Loot use attribute level and methods like rand, setLevel and getLevel. Attribute level used to relate the enemy and loot as the enemy and loot exist in all stages. For method rand, it is used to randomize the health for the enemy and loot for each stage.

4. *Polymorphism relationship*

I. Class Enemy and Class Loot

Class Enemy and Class Loot have a polymorphism relation because they used the same method which is winnerHp but different outcome. For instance, when the enemy fights with a player, the health is determined by condition, which is if the player 's health is less than the enemy, the method will return the value zero which means zero health. If the player's health is more than the enemy, the enemy's health will add to the player's health and return the added health. However, the loot health will be added to the player without any condition.

3.0 UML Diagram

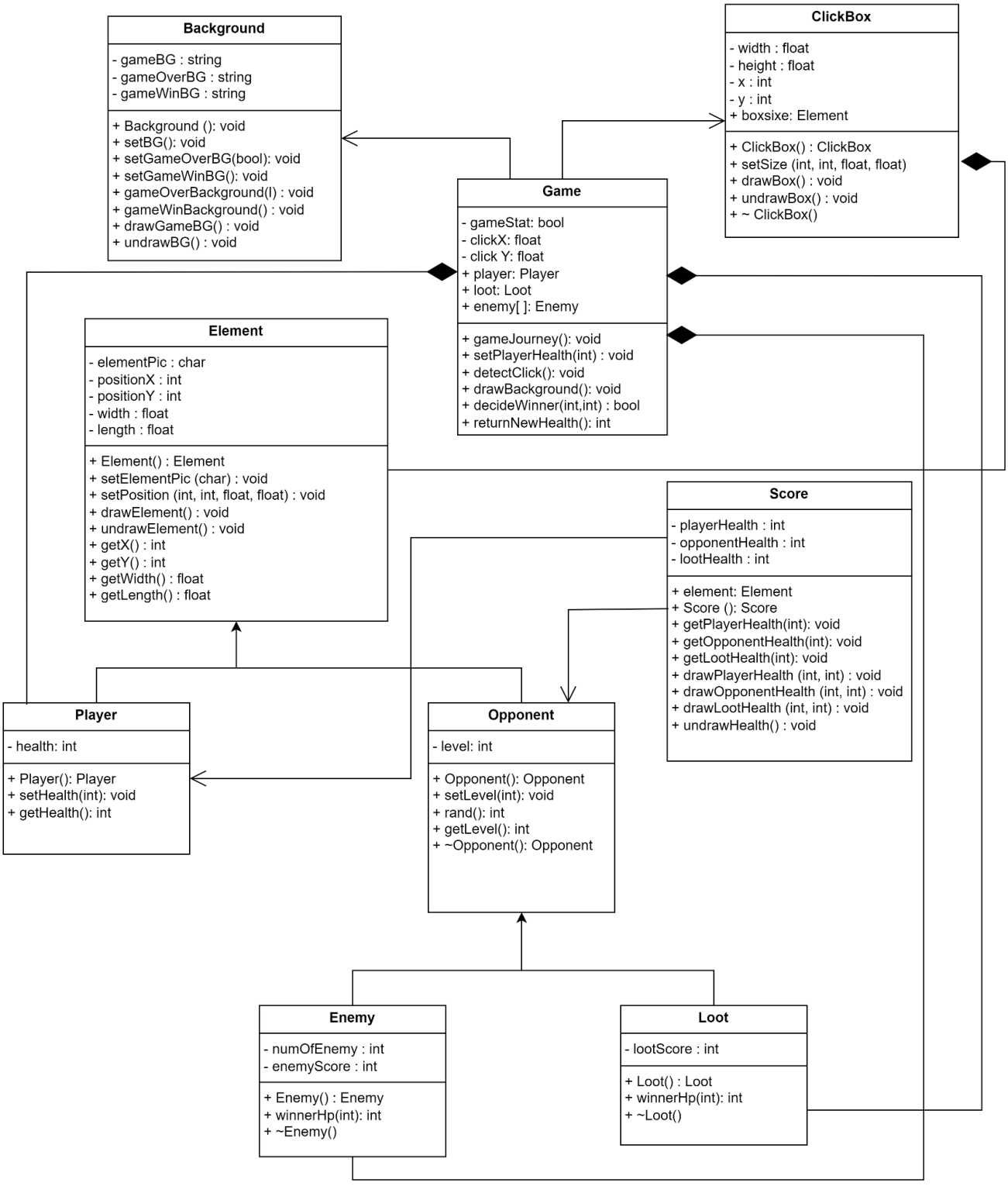

Figure 1: UML Diagram for Math Battle